

RS003 — Gemini Art Guide: 4 Ships + 1 Background

Topic: Making Your Own Game Art PNGs with Google Gemini · **Course:** Pygame Space Shooter ·

Time: about 45 minutes

Follow these steps exactly and you finish with **5 PNG files**: 1 player ship, 3 enemy ships, 1 space background. Then your game stops using code-drawn shapes and starts using **your** art.

 Open Gemini with your teacher or a grown-up. Look at every picture before you use it.

1 · Set Up

1. Go to **gemini.google.com** and sign in with a grown-up.
2. Start a **new chat**. Click the **image** button (it says *Create image*, or you pick the image tool from the  menu).
3. Find your game folder — the folder holding your  file.
4. Inside it, make a new folder named exactly:

```
art
```

Every picture you make goes in . Lower case. No spaces.

 One prompt = one picture. Never ask for all 4 ships in one go — you get one messy picture you cannot use.

2 · The Magenta Trick 🎩

Your ship must sit on **your** space background, not inside a white box.

Gemini is not good at making see-through pictures. So you do this instead: ask for a **bright magenta** background, then tell Pygame "*magenta means invisible*". Magenta is that shocking pink —

`#FF00FF`. You pick it because **no ship is ever that colour**, so nothing else disappears by accident.

Every ship prompt below already says magenta. Do not change that line.

The background picture is the opposite — it fills the whole screen, so it has **no** magenta.

3 · Prompt 1 · Player Ship 🚀

Paste this into Gemini exactly as written:

```
A small player spaceship for a 2D arcade game, 32-bit pixel art,
retro arcade style, top-down view seen from directly above, nose
pointing up, blue and white with a glowing engine, hard clean
edges, no glow spill, no shadow, centred, on a solid bright
magenta background (#FF00FF), one single sprite, no text.
```

When the picture appears, check it against \$7. If it is good, save it as:

```
player.png
```

Want a different look? Change **only** the colour words (`blue and white`) or the ship words (`small player spaceship`). Everything after `hard clean edges` stays the same every time.

4 · Prompts 2–4 · The Three Enemy Ships 🚀

Same recipe, three times. Each enemy gets its own **shape word** and **colour** so players can tell them apart in a fast fight.

Enemy 1 — the spiky one

A small spiky alien enemy fighter for a 2D arcade game, 32-bit pixel art, retro arcade style, top-down view seen from directly above, nose pointing down, green and yellow, sharp pointed wings, hard clean edges, no glow spill, no shadow, centred, on a solid bright magenta background (#FF00FF), one single sprite, no text.

Save as `enemy1.png`

Enemy 2 — the round one

A small round alien enemy saucer for a 2D arcade game, 32-bit pixel art, retro arcade style, top-down view seen from directly above, purple and pink, smooth circular body with one glowing eye, hard clean edges, no glow spill, no shadow, centred, on a solid bright magenta background (#FF00FF), one single sprite, no text.

Save as `enemy2.png`

Enemy 3 — the heavy one

A big heavy alien enemy battleship for a 2D arcade game, 32-bit pixel art, retro arcade style, top-down view seen from directly above, nose pointing down, dark red and grey, thick armour plates, hard clean edges, no glow spill, no shadow, centred, on a solid bright magenta background (#FF00FF), one single sprite, no text.

Save as `enemy3.png`

💡 **Enemies point down, your player points up.** They fly at you. If Gemini draws it the wrong way round, say: *"same ship, nose pointing down"*.

5 · Prompt 5 · The Background 🌌

This one is different — **no magenta**, and it fills the whole picture.

A deep space background for a 2D arcade game, 32-bit pixel art, retro arcade style, dark navy blue night sky, tiny scattered stars, one distant purple planet in the corner, faint nebula clouds, seamless flat wide background, top-down view, empty middle, no ships, no people, no text.

Save as `space.png`

Why empty middle ? Your ships fly there. A busy middle makes them impossible to see.

Why dark ? Bright ships must pop against it. A pale background eats your art.

6 · Save the PNGs

For each picture:

1. Click the picture in Gemini.
2. Click the **download** button (the ↓ arrow).
3. It lands in your **Downloads** folder with a long messy name.
4. **Rename** it and **move** it into your **art** folder.

Your **art** folder must end up looking **exactly** like this — same spelling, same lower case, all **.png**:

```
art/player.png  
art/enemy1.png  
art/enemy2.png  
art/enemy3.png  
art/space.png
```

⚠ If a file downloads as **.jpg** or **.webp**, it will **not** keep magenta clean. Ask Gemini: *"give me this as a PNG"*, or download it again from the image view.

7 · Check Every Picture

Look at each ship before you keep it. Say yes to all four:

- The background is **flat magenta** — one solid colour, no pattern, no fade.
- There is **one** ship — not two, not a fleet.
- You see it **from above** — not the side, not a 3D angle.
- There is **no text**, no letters, no watermark.

When it goes wrong, change **one word** and send the prompt again:

What you see	What to add or change
White or grey box behind ship	<i>"solid bright magenta background #FF00FF, nothing else"</i>
Pink fuzzy halo round the ship	<i>"hard pixel edges, no anti-aliasing, no glow, no shadow"</i>
Ship seen from the side	<i>"top-down view, seen from directly above"</i>
Two or more ships	<i>"exactly one ship, centred"</i>
Looks like a photo, not pixels	<i>"32-bit pixel art, retro arcade, flat colours"</i>
Ship too small in the picture	<i>"ship fills the frame"</i>

Changing one word at a time **is** prompt engineering. That is the real skill this week.

8 · Load Your Art in Pygame 🎮

Put this **after** `pygame.display.set_mode(...)` and **before** your game loop:

```
# background – fills the screen, no magenta
space = pygame.image.load("art/space.png").convert()
space = pygame.transform.scale(space, (WIDTH, HEIGHT))

# ships – magenta becomes invisible
MAGENTA = (255, 0, 255)

player_img = pygame.image.load("art/player.png").convert()
player_img.set_colorkey(MAGENTA)
player_img = pygame.transform.scale(player_img, (64, 64))

enemy1_img = pygame.image.load("art/enemy1.png").convert()
enemy1_img.set_colorkey(MAGENTA)
enemy1_img = pygame.transform.scale(enemy1_img, (48, 48))
```

Then swap your drawing lines. Old shape code out, picture code in:

```
screen.blit(space, (0, 0))          # instead of screen.fill(BLACK)
screen.blit(player_img, (player_x, player_y))
```

Rules that break your game if you skip them:

- `.convert()` for magenta ships — **not** `.convert_alpha()`. Alpha ignores the colour key and you keep the pink box.
- `set_colorkey` goes **before** `transform.scale`.
- If a picture already came out truly see-through (grey checkerboard), use `.convert_alpha()` and **no** colorkey line.
- Load pictures **once**, at the top. Loading inside the game loop makes the game crawl.

9 · Show Your Work 📷 🎥

Put your 5 pictures into **art**, load them into your game, and run it.

Record **one video** — one take, no stopping (a phone is fine). Show these in order:

1. Your 5 pictures, one at a time.
2. One prompt read out loud — point to the magenta line and say what it does.
3. One picture Gemini got wrong, and the word you changed to fix it.
4. Your game running with your own art.

Fill the blanks:

Today I made _____.

I built it using this code: _____.

In this code I used _____.

The hardest part was _____.

That part was hard because _____.

The most fun part was _____.

Something new I learned was _____.

Submit 

Send your **5 PNG files** + your video to teacher on **KakaoTalk**.